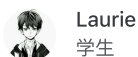




深入理解 RoPE（Rotary Position Embedding）旋转位置编码



Laurie
学生

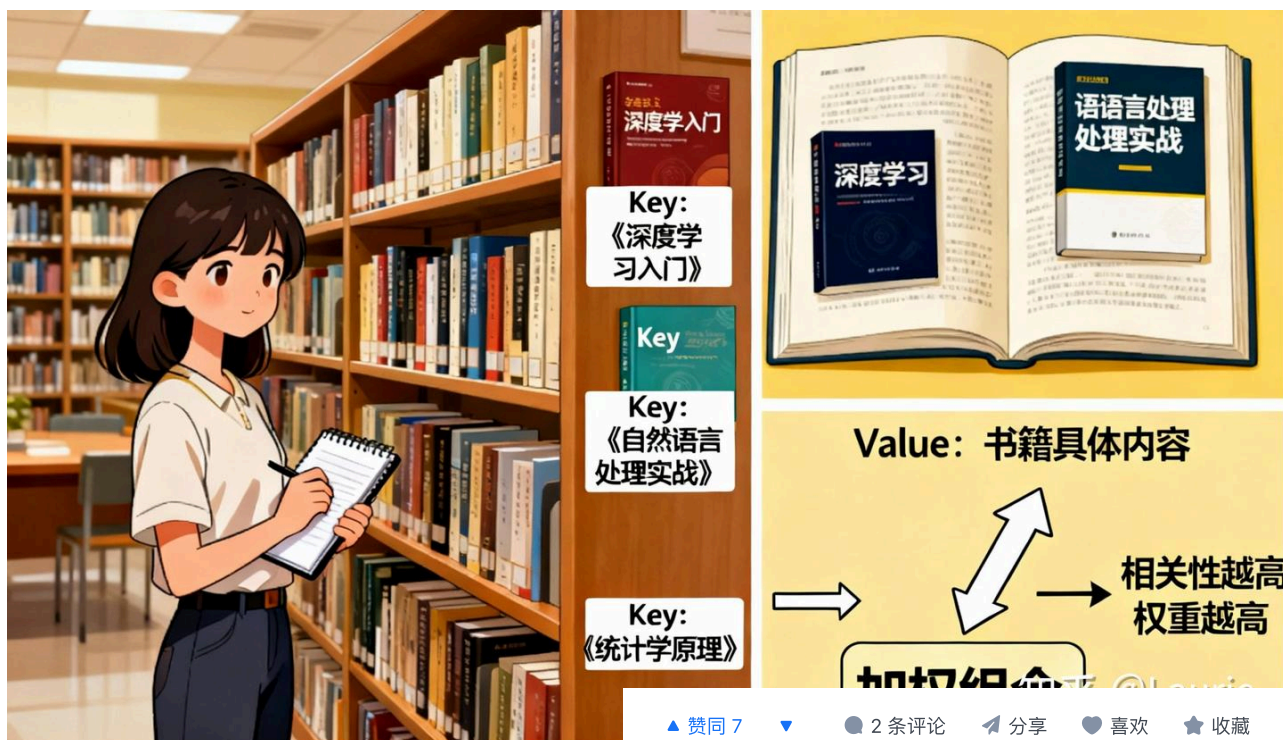
7 人赞同了该文章 >

一个 token 在序列中的第 5 个位置，和在第 500 个位置，模型应该如何区分？更重要的是，相距 10 个位置的两个词，无论它们出现在序列开头还是结尾，这种“相隔 10 步”的关系应该保持一致。传统位置编码给每个位置贴上固定标签，却难以优雅地捕捉这种相对关系。RoPE 的答案是：让向量在复平面⁺上旋转，用旋转角度编码位置，用角度差编码距离。这个看似抽象的数学操作，为何成为了 LLaMA、Qwen 等主流大模型的共同选择？本文将带你一探究竟。

一、从 Attention 机制说起

在深入 RoPE 之前，我们必须首先牢固地掌握 Self-Attention 机制，特别是它为何需要位置编码。

二、核心比喻：在图书馆查资料



赞同 7



2 条评论



分享



喜欢



收藏



申请转



想象一下你在一个大型图书馆里，想找关于“**大语言模型+**”的资料。

- **Query (Q) - 你的查询问题**

你脑海中带着一个具体的问题，比如“什么是 Transformer 模型？”。这个问题就是你的 Query。

- **Key (K) - 书架上每本书的标签或标题**

图书馆里有成千上万本书，你不可能每一本都从头读到尾。

你会快速浏览每本书的**标题或标签 (Key)**，比如《深度学习入门》、《**自然语言处理+**实战》、《统计学原理》等等。

你会用你的 **问题 (Query)** 去和这些**标签 (Key)** 进行匹配，看看哪个标签和你的问题最相关。例如，《深度学习入门》这个 Key 和你的 Query “Transformer 模型”就很相关，而《统计学原理》这个 Key 相关性就低一些。

- **Value (V) - 书本的具体内容**

一旦你通过 Q 和 K 的匹配，找到了最相关的几本书（比如你发现《深度学习入门》和《自然语言处理实战》这两本书的 Key 最匹配你的 Query），你就会去阅读这几本书的**具体内容 (Value)**。

你最终获得的知识，不是这些书的标题，而是它们内容的**加权组合**。相关性越高的书，你在最终形成的答案中赋予它的权重就越高。

总结这个比喻：

Attention 机制 ≈ 在图书馆通过“查询(Q)”匹配“书籍标签(K)”，然后根据匹配度去“加权阅读书籍内容(V)”，最终得到你想要的答案。

三、技术解释：Q, K, V 在做什么

在深度学习模型（尤其是 Transformer）中，Q, K, V 都是**向量 (Vector)**。它们通常是由同一个输入向量（比如一个词的嵌入向量 x ）通过不同的线性变换（乘以不同的权重矩阵）得到的。

假设我们有一个输入词向量 x ：

1. Query (Q): $Q = xW_q$

2. Key (K): $K = xW_k$

3. Value (V): $V = xW_v$

这里的 W_q , W_k , W_v 是三个独立的、需要通过模型训练学习的权重矩阵。

为什么需要三个不同的矩阵？

因为它们扮演着不同的角色，将原始输入信息投影到三个不同的功能空间：

- W_q 将输入 x 转换成一个“查询”向量，它代表 x 要去“问问题”、“寻找关联”的能力。
- W_k 将输入 x 转换成一个“键”向量，它代表 x “被查询”、“被关联”的特性。
- W_v 将输入 x 转换成一个“值”向量，它代表 x 自身所携带的、要被提取的**真正信息**。

四、Attention 的计算步骤

现在我们把图书馆的例子和技术步骤结合起来。假设我们要计算句子“The cat sat on the mat”中，“sat”这个词的上下文感知表示。

输入：句子中每个词的词向量 (x_{the} , x_{cat} , x_{sat} , ...)。

目标：为“sat”生成一个新的、包含了上下文信息的向量。

详细步骤

1. 生成 Q, K, V:

我们主要关注“sat”这个词，所以用它的词向量 x_{sat} 来生成 Query:

$$Q_{\text{sat}} = x_{\text{sat}} W_q$$

句子中的每一个词（包括 “sat” 自己）都需要扮演“被查询”的角色，所以每个词的向量 x_i 都需要生成自己的 Key 和 Value：

$$\begin{aligned}K_{\text{the}} &= x_{\text{the}} W_k, & V_{\text{the}} &= x_{\text{the}} W_v \\K_{\text{cat}} &= x_{\text{cat}} W_k, & V_{\text{cat}} &= x_{\text{cat}} W_v \\K_{\text{sat}} &= x_{\text{sat}} W_k, & V_{\text{sat}} &= x_{\text{sat}} W_v \\&\dots\end{aligned}$$

1. 计算注意力分数 (Attention Score)：

用 “sat” 的 Query (Q_{sat}) 和句子中所有词的 Key (K_i) 进行点积运算，来计算相关性。

$$\begin{aligned}\text{score}_{\text{the}} &= Q_{\text{sat}} \cdot K_{\text{the}} \\ \text{score}_{\text{cat}} &= Q_{\text{sat}} \cdot K_{\text{cat}} \\ \text{score}_{\text{sat}} &= Q_{\text{sat}} \cdot K_{\text{sat}} \\ &\dots\end{aligned}$$

分数越高，代表那个词和 “sat” 的关系越紧密。在这个例子里，我们期望 $\text{score}_{\text{cat}}$ 会比较高。

1. 归一化 (Softmax)：

将上一步得到的所有分数进行 Softmax 操作，得到一组权重 (weights)，这组权重的和为 1。

$$\text{weights} = \text{softmax}([\text{score}_{\text{the}}, \text{score}_{\text{cat}}, \text{score}_{\text{sat}}, \dots])$$

例如，可能得到 $\text{weights} = [0.2, 0.6, 0.1, \dots]$ ，这表示模型认为 “cat” 与 “sat” 的关系权重为 0.6，最为重要。

1. 加权求和：

用上一步得到的权重，去加权求和所有词的 Value (V) 向量。

$$\text{output}_{\text{sat}} = (\text{weight}_{\text{the}} \cdot V_{\text{the}}) + (\text{weight}_{\text{cat}} \cdot V_{\text{cat}}) + (\text{weight}_{\text{sat}} \cdot V_{\text{sat}}) + \dots$$

或简写为：

$$\text{output}_{\text{sat}} = 0.2V_{\text{the}} + 0.6V_{\text{cat}} + 0.1V_{\text{sat}} + \dots$$

最终结果：

$\text{output}_{\text{sat}}$ 就是 “sat” 这个词经过 Attention 机制之后新的表示。这个新的向量不仅包含了 “sat” 本身的信息，还重点融入了 “cat” 的信息以及少量其他词的信息。它变得 上下文感知 (context-aware) 了。

五、从单步计算到矩阵公式：Attention 公式的诞生

在第三部分，我们以 “sat” 这个词为例，详细描述了如何为它计算出上下文感知的向量。但模型在实际运行时，需要一次性为句子中的所有词都计算出新的表示。如果为每个词都走一遍循环，效率会非常低下。

幸运的是，深度学习中的所有操作都可以被优雅地转换成矩阵运算，从而利用 GPU 进行大规模并行计算。现在，我们把第三部分的操作“打包”成矩阵形式。

5.1. 输入打包成矩阵

假设我们的输入句子有 n 个词，每个词的词向量维度是 d_{model} 。我们可以把所有词向量堆叠起来，形成一个输入矩阵 X 。

$$X = \begin{pmatrix} \mathbf{x}_{the} \\ \mathbf{x}_{cat} \\ \mathbf{x}_{sat} \\ \vdots \end{pmatrix}$$

这个 X 矩阵的维度是 (n, d_{model}) 。

5.2. Q, K, V 的矩阵化计算

我们不再为每个词单独计算 Q, K, V，而是用输入矩阵 X 与权重矩阵 W_q, W_k, W_v 直接相乘，一次性得到所有词的 Q, K, V 向量组成的矩阵。

假设 Q, K 向量的维度是 d_k ，V 向量的维度是 d_v （通常 $d_k = d_v$ ，分开定义是为了灵活性）。

$$Q = XW_q \quad (\text{维度: } (n, d_k))$$

$$K = XW_k \quad (\text{维度: } (n, d_k))$$

$$V = XW_v \quad (\text{维度: } (n, d_v))$$

现在， Q 矩阵的第 i 行就是第 i 个词的 Query 向量， K 矩阵的第 j 行就是第 j 个词的 Key 向量，以此类推。

5.3. 注意力分数的矩阵化计算

回顾一下，计算“sat”（第 i 个词）的注意力分数，我们需要用它的 Q_i 去和所有词的 K_j 进行点积。

在矩阵中，这相当于用 Q 矩阵的第 i 行，去和 K 矩阵的每一行做点积。这可以通过将 K 矩阵转置 (K^T) 然后与 Q 矩阵相乘来实现。

$$\text{Scores} = QK^T$$

我们来验证一下这个操作：

- Q 的维度是 (n, d_k)
- K^T 的维度是 (d_k, n)
- QK^T 的结果是一个 (n, n) 的矩阵。

这个 (n, n) 矩阵的第 i 行第 j 列的元素，正好就是第 i 个词的 Query 向量 (Q_i) 与第 j 个词的 Key 向量 (K_j) 的点积。这完美地实现了我们想要的一次性计算所有词对之间的相关性分数！

5.4. 引入缩放因子 (Scaling)

3.2.1 Scaled Dot-Product Attention

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k . We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $1/\sqrt{d_k}$.

在“Attention Is All You Need”论文中，作者发现当 d_k 的维度较大时，点积 $Q \cdot K$ 的结果的方差也会增大，这可能导致数值过大。当这些很大的数值进入 Softmax 函数后，会使得梯度变得非常小，不利于模型训练。

为了缓解这个问题，他们提出在送入 Softmax 之前，将分数除以一个缩放因子，这个因子就是 $\sqrt{d_k}$ 。

$$\text{Scaled Scores} = \frac{QK^T}{\sqrt{d_k}}$$

这便是著名的缩放点积注意力 (Scaled Dot-Product Attention) 中“缩放” (Scaled) 的由来。

5.5. Softmax 和加权求和

接下来的步骤和之前完全一样，只是操作对象变成了整个矩阵：

- Softmax：对 $\frac{QK^T}{\sqrt{d_k}}$ 矩阵按行进行 Softmax 归一化，得到注意力权重矩阵 A （维度仍为 (n, n) ）。矩阵 A 的第 i 行，就是第 i 个词对句子中其他所有词的注意力权重。
- 加权求和：将得到的权重矩阵 A 与 V 矩阵相乘。

$$\text{Output} = A \cdot V = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- A 的维度是 (n, n)
- V 的维度是 (n, d_v)
- 最终输出 Output 的维度是 (n, d_v) 。

这个输出矩阵的第 i 行，正是我们之前费力计算的 output_i （例如 $\text{output}_{\text{sat}}$ ），它是一个融合了整个句子上下文信息的新向量。

5.6. 最终的 Attention 公式

通过将所有步骤矩阵化，我们最终得到了那个简洁而强大的 Scaled Dot-Product Attention 公式：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

这个公式优雅地概括了整个 Attention 机制的核心：

1. QK^T ：计算所有查询（Query）与所有键（Key）之间的相似度。
2. $\frac{\dots}{\sqrt{d_k}}$ ：进行缩放，稳定训练。
3. $\text{softmax}(\dots)$ ：将相似度分数转换成权重。
4. $\dots V$ ：用得到的权重对所有值（Value）进行加权求和，得到最终的输出。

从一个直观的图书馆比喻，到为单个词分步计算，再到最终推导出这个并行的矩阵公式，我们完整地理解了 Attention 机制的“是什么”、“为什么”和“怎么做”。

六、Attention 机制的“盲点”：位置信息的缺失

到目前为止，我们已经建立了一个强大的机制，它可以让模型在处理一个词时，动态地关注句子中的其他所有词。这使得模型能够捕捉到词与词之间的依赖关系，比如 “sat” 应该重点关注 “cat”。

然而，这个机制有一个固有的、严重的缺陷：它是一个“无序”的模型。

想象一下，你把一袋子写着单词的纸条倒在桌子上。Attention 机制可以告诉你哪些词条应该和哪些词条放在一起（比如 “cat” 和 “sat”），但它完全不知道这些词条原来的排列顺序。

对于语言来说，顺序至关重要。考虑这两个句子：

- “狗咬人” (Dog bites man) - 一个常见事件。
- “人咬狗” (Man bites dog) - 一条大新闻。

这两个句子使用了完全相同的词汇，但由于顺序不同，它们的语义天差地别。如果我们的模型无法区分这两种情况，那它就无法真正理解语言。

那么，我们目前推导出的 Scaled Dot-Product Attention 公式能否区分这两种情况呢？

答案是：不能。

6.1. 数学上的直觉证明

我们回顾一下最终的输出是如何计算的。对于句子中的第 i 个词，其输出向量 output_i 是所有词的 Value 向量 V_j 的加权和：

$$\text{output}_i = \sum_{j=1}^n \text{weight}_{ij} \cdot V_j$$

这里的权重 weight_{ij} 来自于 Q_i 和 K_j 的点积。

关键在于，这个求和操作 ($\sum$) 是顺序无关的 (commutative)。 $a \cdot V_1 + b \cdot V_2$ 和 $b \cdot V_2 + a \cdot V_1$ 的结果是完全一样的。

现在，让我们思考“狗咬人”和“人咬狗”这两个句子中，对于“咬”这个词的计算过程：

- 在“狗咬人”中：
 - “咬”的 Query ($Q_{\text{咬}}$) 会分别与“狗”的 Key ($K_{\text{狗}}$)、“咬”的 Key ($K_{\text{咬}}$)、“人”的 Key ($K_{\text{人}}$) 计算注意力分数。
 - 最终“咬”的输出是 $w_1 V_{\text{狗}} + w_2 V_{\text{咬}} + w_3 V_{\text{人}}$ 的加权和。
- 在“人咬狗”中：
 - “咬”的 Query ($Q_{\text{咬}}$) 仍然是与“人”的 Key ($K_{\text{人}}$)、“咬”的 Key ($K_{\text{咬}}$)、“狗”的 Key ($K_{\text{狗}}$) 计算分数。
 - 注意，因为 Q, K, V 都是由词向量本身唯一决定的，所以 $Q_{\text{咬}}, K_{\text{狗}}, V_{\text{人}}$ 等等这些向量的值是不变的，无论它们出现在句子的哪个位置。
 - 因此，“咬”的最终输出依然是基于同样一组 $\{V_{\text{狗}}, V_{\text{咬}}, V_{\text{人}}\}$ 向量的加权和，权重也来自同样一组 Key 向量。

因为求和的顺序不影响最终结果，所以模型计算出的“咬”的上下文表示在这两个句子中将是完全相同的。同理，“人”和“狗”的输出表示也无法区分它们在句子中的角色（是主语还是宾语）。

这个模型本质上将句子视为一个词袋 (Bag of Words)，丢失了至关重要的顺序信息。

6.2. 代码实例证明

口说无凭，我们用代码来验证这个结论。我们将使用 PyTorch 来模拟 Attention 的计算过程。

```
import torch
import torch.nn.functional as F
import numpy as np

# --- 1. 环境设置 ---
# 为了结果可复现，设置随机种子
torch.manual_seed(42)
np.random.seed(42)

# 定义词汇表和词向量 (Embedding)
vocab = {"<pad>": 0, "人": 1, "咬": 2, "狗": 3}
# d_model = 8 (嵌入维度)
embeddings = torch.nn.Embedding(num_embeddings=len(vocab), embedding_dim=8)

# 两个语义完全不同的句子
sentence1 = ["人", "咬", "狗"]
sentence2 = ["狗", "咬", "人"]

# 将句子转为 token ID
tokens1 = torch.tensor([vocab[word] for word in sentence1]) # tensor([1, 2, 3])
tokens2 = torch.tensor([vocab[word] for word in sentence2]) # tensor([3, 2, 1])

# 获取词向量
x1 = embeddings(tokens1) # 句子1的输入矩阵 X
x2 = embeddings(tokens2) # 句子2的输入矩阵 X

# --- 2. 模拟 Attention 计算 ---
d_model = 8 # 输入维度
d_k = 16 # Q, K 维度
d_v = 16 # V 维度

# 定义共享的权重矩阵 Wq, Wk, Wv
W_q = torch.randn(d_model, d_k)
W_k = torch.randn(d_model, d_k)
```

```

W_v = torch.randn(d_model, d_v)

# 定义一个函数来执行 Attention 计算
def scaled_dot_product_attention(X, W_q, W_k, W_v):
    Q = X @ W_q
    K = X @ W_k
    V = X @ W_v

    d_k = K.shape[-1]
    scores = Q @ K.transpose(-2, -1) / np.sqrt(d_k)
    attention_weights = F.softmax(scores, dim=-1)
    output = attention_weights @ V
    return output

# --- 3. 计算并对比结果 ---
output1 = scaled_dot_product_attention(x1, W_q, W_k, W_v)
output2 = scaled_dot_product_attention(x2, W_q, W_k, W_v)

print("句子1 ('人', '咬', '狗') 的 Attention 输出:")
print(output1)
print("\n句子2 ('狗', '咬', '人') 的 Attention 输出:")
print(output2)

print("\n--- 结果分析 ---")
# 提取“人”在两个句子中的输出向量
# 在句子1中，“人”是第0个词
output_ren_1 = output1[0]
# 在句子2中，“人”是第2个词
output_ren_2 = output2[2]

# 提取“狗”在两个句子中的输出向量
# 在句子1中，“狗”是第2个词
output_gou_1 = output1[2]
# 在句子2中，“狗”是第0个词
output_gou_2 = output2[0]

# 检查向量是否相等
are_ren_vectors_equal = torch.allclose(output_ren_1, output_ren_2)
are_gou_vectors_equal = torch.allclose(output_gou_1, output_gou_2)

print(f"“人”在两个句子中的输出向量是否相同? {are_ren_vectors_equal}")
print(f"“狗”在两个句子中的输出向量是否相同? {are_gou_vectors_equal}")

```

代码运行结果：

```

句子1 ('人', '咬', '狗') 的 Attention 输出:
tensor([[ 2.7134, -3.1083, -0.1133,  1.5497,  2.6582, -0.2903, -1.5191, -1.0143,
          2.8338, -5.3177,  2.1151,  4.7673,  1.3531,  0.3049,  1.0815,  0.9538],
        [ 2.9598, -3.2394,  0.6187, -0.3445,  2.2826,  0.2870, -2.6724, -1.4579,
          3.5696, -3.4276,  1.5118,  3.2930,  0.9988, -0.1089,  1.5917,  0.2262],
        [ 2.8089, -3.1568,  0.1227,  0.9663,  2.5538, -0.1277, -1.8818, -1.1403,
          3.0872, -4.7199,  1.9279,  4.3406,  1.2569,  0.1728,  1.2330,  0.7427]],
        grad_fn=<MmBackward0>)

句子2 ('狗', '咬', '人') 的 Attention 输出:
tensor([[ 2.8089, -3.1568,  0.1227,  0.9663,  2.5538, -0.1277, -1.8818, -1.1403,
          3.0872, -4.7199,  1.9279,  4.3406,  1.2569,  0.1728,  1.2330,  0.7427],
        [ 2.9598, -3.2394,  0.6187, -0.3445,  2.2826,  0.2870, -2.6724, -1.4579,
          3.5696, -3.4276,  1.5118,  3.2930,  0.9988, -0.1089,  1.5917,  0.2262],
        [ 2.7134, -3.1083, -0.1133,  1.5497,  2.6582, -0.2903, -1.5191, -1.0143,
          2.8338, -5.3177,  2.1151,  4.7673,  1.3531,  0.3049,  1.0815,  0.9538]],
        grad_fn=<MmBackward0>)

--- 结果分析 ---

```

“人”在两个句子中的输出向量是否相同？ True

“狗”在两个句子中的输出向量是否相同？ True

结论一目了然：

- 句子1 output1 的第1行（对应“人”）和句子2 output2 的第3行（也对应“人”）的向量是完全一样的。
- 同样，句子1 output1 的第3行（对应“狗”）和句子2 output2 的第1行（也对应“狗”）的向量也是完全一样的。

即使字是相同的，但是“主语”和“宾语”这一层关系是不一样的。这清晰地证明了，标准的 Self-Attention 机制是位置不敏感的。它只关心“句子中有什么”，而不关心“它们在哪里”。

七、在 Attention 机制中引入位置信息

这种对顺序的“失明”是无法接受的。为了让模型能够理解序列的顺序，我们必须找到一种方法，将位置信息注入到模型中。

目标：我们希望 Attention 分数，即 Query 向量 $\mathbf{q}$ 和 Key 向量 $\mathbf{k}$ 的点积 $\langle \mathbf{q}, \mathbf{k} \rangle$ ，能够体现它们之间的位置信息。然而，位置信息有多种形式，比如绝对位置、相对位置、相对距离等，我们希望找到一种最简单、最有效的方式来实现这一点。

具体来说，假设一个 Query 向量 $\mathbf{q}$ 在序列的第 m 个位置，一个 Key 向量 $\mathbf{k}$ 在序列的第 n 个位置。我们希望通过某种函数 f 将位置信息融合⁺进这两个向量，得到 $\mathbf{q}' = f(\mathbf{q}, m)$ 和 $\mathbf{k}' = f(\mathbf{k}, n)$ 。我们对这个融合后的点积 $\langle \mathbf{q}', \mathbf{k}' \rangle$ 有一个核心期望：它应该只与相对位置 $(m - n)$ 有关。

为什么是相对位置？因为“相距 10 步”这个关系，无论是在句子开头（第 5 个词和第 15 个词）还是在句子结尾（第 500 个词和第 510 个词），其语义上的关联强度应该是相似的。模型不应该因为绝对位置的变化而改变对这种相对关系的判断。

八、传统位置编码的局限性

在 RoPE 出现之前，主流的位置编码方案⁺主要有两种：

1. 绝对位置编码 (Sinusoidal Positional Encoding)：这是“Attention Is All You Need”论文中提出的经典方案。

- 方法：为每个绝对位置 $(0, 1, 2, \dots)$ 生成一个固定的、独特的向量（通常是正弦和余弦⁺函数生成的），然后直接加到词向量上。即 $\mathbf{x}' = \mathbf{x}_{\text{embedding}} + \mathbf{p}_m$ 。
- 优点：实现简单，且不增加任何可训练参数。
- 缺点：
 - 相对位置关系不直观：两个位置的编码向量相加，其结果并不能很好地表示它们的相对位置关系。模型需要自己从“绝对位置的和”中去学习“相对位置”的概念，这增加了学习难度。
 - 外推性差 (Poor Extrapolation)：如果模型训练时最大序列长度是 8192，那么它从未见过位置 8193 的编码。当需要处理更长的序列时，模型表现会急剧下降。

1. 可学习的绝对位置编码 (Learned Absolute Positional Encoding)：BERT 等模型采用的方案。

- 方法：创建一个位置编码矩阵，其大小为 $(\text{max_sequence_length}, \text{embedding_dim})$ 。这个矩阵是可训练的参数。在输入时，根据词的位置，从这个矩阵中查找对应的位置向量⁺并加到词向量上。
- 优点：更加灵活，模型可以自己学习出最适合任务的位置表示。
- 缺点：
 - 外推性更差：它完全无法处理超过 $\text{max_sequence_length}$ 的序列，因为根本没有为这些新位置预留参数。
 - 参数量增加：引入了额外的可训练参数。

这两种方法都通过“加法”将位置信息硬塞进词向量里，但它们都不能优雅地解决相对位置的表示问题。RoPE 正是为了解决这一核心痛点而设计的。

九、RoPE 的核心思想：旋转即位置

RoPE 的全称是旋转位置编码 (Rotary Position Embedding)。它的核心思想极其巧妙：通过向量的旋转来编码位置信息。

让我们回到文章开头的比喻：

让向量在复平面上旋转，用旋转角度编码位置，用角度差编码距离。

具体来说，RoPE 认为，一个词的 Query 向量或 Key 向量，它在不同位置的表示，应该是其原始表示经过旋转得到的。

- 绝对位置：由一个固定的旋转角度来表示。例如，位置 m 对应旋转角度 $m\theta$ 。
- 相对位置：两个向量之间的旋转角度差。例如，位置 m 和位置 n 的向量，它们之间的相对位置关系体现在角度差 $(m - n)\theta$ 上。

这个设计的绝妙之处在于，向量点积与旋转操作具有天然的相容性。两个向量的点积结果，在它们被同步旋转之后，其结果只与它们之间的夹角差（相对旋转）有关，而与它们各自的绝对旋转角度无关。这正是我们梦寐以求的“只依赖于相对位置”的特性！

十、从数学原理*到实现：两种视角下的同一座山峰

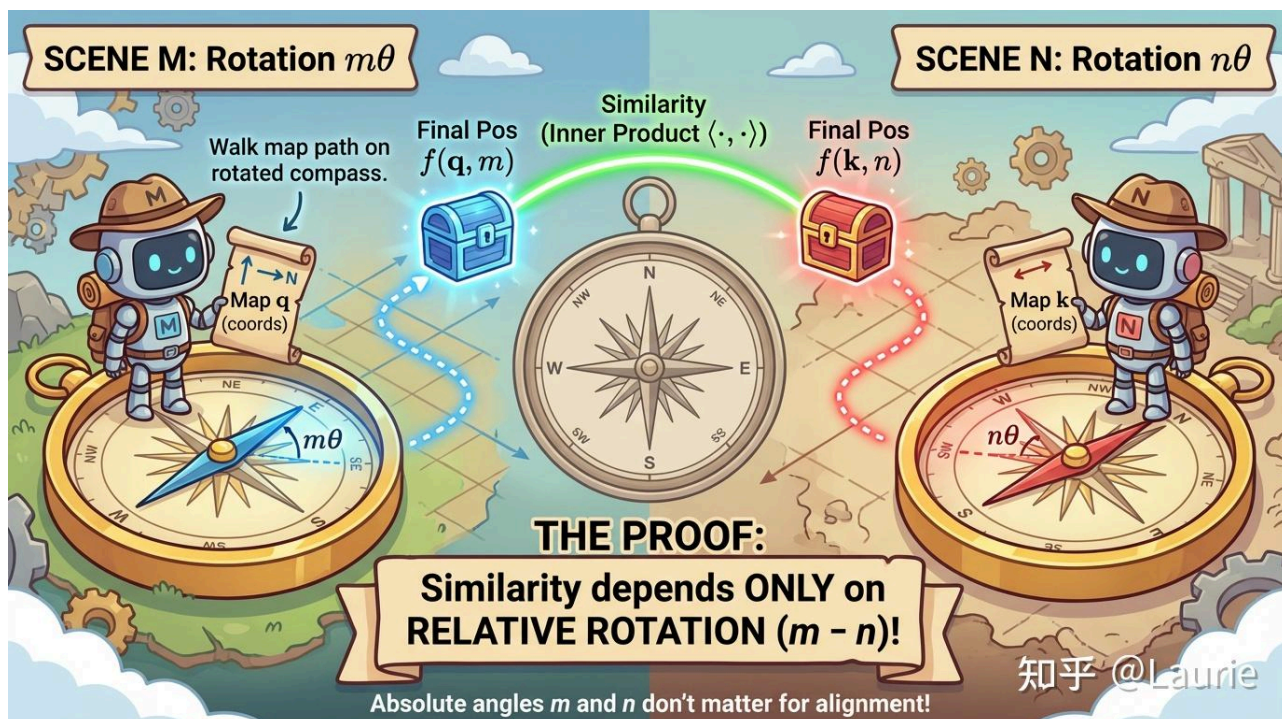
在 RoPE（旋转位置编码）的论文中，我们追求一个核心目标：让任意两个 token 的注意力得分，只与它们的相对位置有关，而与它们的绝对位置无关。用数学语言来说，就是对于任意两个向量 $\mathbf{q}$ 和 $\mathbf{k}$ ，它们在经过位置 m 和 n 的编码后，其内积结果必须可以表示成一个只依赖于 $(m - n)$ 的函数。

$$\langle f(\mathbf{q}, m), f(\mathbf{k}, n) \rangle = g(\mathbf{q}, \mathbf{k}, m - n)$$

RoPE 的实现方式是“旋转”，但证明这个“旋转”能满足上述性质，却有两条截然不同的路径。一条是崎岖泥泞的盘山路，另一条则是穿越山体的优雅隧道。让我们先踏上这条盘山路，感受一下它的挑战。

引入比喻：寻宝之旅

为了让整个过程更形象，我们引入一个比喻：



map

- 向量 $\mathbf{q}$ 和 $\mathbf{k}$ ：想象这是两张藏宝图，上面用二维坐标记录了从“起点”到“宝藏”的路线。例如， $\mathbf{q} = (q_1, q_2)$ 就是一张图，指示你“向东走 q_1 步，再向北走 q_2 步”。
- 位置 m 和 n ：想象你站在一个巨大的、可以旋转的罗盘上。位置 m 和 n 代表罗盘的两个不同初始旋转角度。比如， $m = 0$ 时罗盘的“北”指向正北方； $m = 1$ 时，罗盘整体顺时针转了 θ 度。
- RoPE 操作 $f(\mathbf{q}, m)$ ：这代表你在一个已经旋转了 $m\theta$ 角度的罗盘上，按照藏宝图 $\mathbf{q}$ 的指示去寻找宝藏。你最终的位置，就是 $f(\mathbf{q}, m)$ 。

- 内积 $\langle \cdot, \cdot \rangle$ ：这可以理解成衡量两个最终宝藏位置的“相似性”或“对齐程度”。如果两个宝藏都在同一个方向上，内积就大；如果方向垂直，内积就为 0。
- 我们的证明目标：证明两个宝藏位置的“相似性”，只取决于两张罗盘的相对旋转角度（即 $m - n$ ），而与它们各自转了多少（ m 和 n 的具体值）无关。

方法一：硬碰硬的三角函数之旅 (旋转矩阵*法)

谈到旋转，大多数人脑海中首先浮现的往往是一个旋转矩阵——它植根于我们熟悉的几何与线性代数*知识，清晰地告诉我们“如何计算”，所以我们先选择这条路。

第一步：定义操作

一个二维向量 $\begin{pmatrix} x \\ y \end{pmatrix}$ 逆时针旋转角度 α ，就是左乘一个旋转矩阵：

$$R_\alpha = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$$

根据我们的“寻宝比喻”，向量 $\mathbf{q} = \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}$ 在位置 m 处，罗盘转了 $m\theta$ 度，所以它旋转后的新位置 $\mathbf{q}'$ 是：

$$\mathbf{q}' = f(\mathbf{q}, m) = R_{m\theta} \mathbf{q} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} q_1 \\ q_2 \end{pmatrix} = \begin{pmatrix} q_1 \cos(m\theta) - q_2 \sin(m\theta) \\ q_1 \sin(m\theta) + q_2 \cos(m\theta) \end{pmatrix}$$

同理，向量 $\mathbf{k} = \begin{pmatrix} k_1 \\ k_2 \end{pmatrix}$ 在位置 n 处的新位置 $\mathbf{k}'$ 是：

$$\mathbf{k}' = f(\mathbf{k}, n) = R_{n\theta} \mathbf{k} = \begin{pmatrix} k_1 \cos(n\theta) - k_2 \sin(n\theta) \\ k_1 \sin(n\theta) + k_2 \cos(n\theta) \end{pmatrix}$$

第二步：开始漫长的计算

现在，我们来计算这两个新位置的内积 $\langle \mathbf{q}', \mathbf{k}' \rangle$ 。内积就是对应分量相乘再相加：

$$\begin{aligned} \langle \mathbf{q}', \mathbf{k}' \rangle &= (q_1 \cos(m\theta) - q_2 \sin(m\theta)) \cdot (k_1 \cos(n\theta) - k_2 \sin(n\theta)) \\ &+ (q_1 \sin(m\theta) + q_2 \cos(m\theta)) \cdot (k_1 \sin(n\theta) + k_2 \cos(n\theta)) \end{aligned}$$

这看起来就像一团乱麻。为了理清它，我们必须硬着头皮把它完全展开：

$$\begin{aligned} &= [q_1 k_1 \cos(m\theta) \cos(n\theta) - q_1 k_2 \cos(m\theta) \sin(n\theta) - q_2 k_1 \sin(m\theta) \cos(n\theta) + q_2 k_2 \sin(m\theta) \sin(n\theta)] \\ &+ [q_1 k_1 \sin(m\theta) \sin(n\theta) + q_1 k_2 \sin(m\theta) \cos(n\theta) + q_2 k_1 \cos(m\theta) \sin(n\theta) + q_2 k_2 \cos(m\theta) \cos(n\theta)] \end{aligned}$$

第三步：整理乱麻——运用三角恒等式*

现在，我们把含有相同 $q_i k_j$ 的项合并在一起：

- 合并 $q_1 k_1$ 项： $q_1 k_1 (\cos(m\theta) \cos(n\theta) + \sin(m\theta) \sin(n\theta))$
- 合并 $q_2 k_2$ 项： $q_2 k_2 (\sin(m\theta) \sin(n\theta) + \cos(m\theta) \cos(n\theta))$
- 合并 $q_1 k_2$ 项： $q_1 k_2 (-\cos(m\theta) \sin(n\theta) + \sin(m\theta) \cos(n\theta))$
- 合并 $q_2 k_1$ 项： $q_2 k_1 (-\sin(m\theta) \cos(n\theta) + \cos(m\theta) \sin(n\theta))$

此时，我们需要从数学工具箱里拿出“和差角公式”：

1. $\cos(A - B) = \cos A \cos B + \sin A \sin B$
2. $\sin(A - B) = \sin A \cos B - \cos A \sin B$

将这些公式应用到我们合并后的项中：

- $q_1 k_1$ 和 $q_2 k_2$ 的系数都变成了 $\cos((m - n)\theta)$ 。
- $q_1 k_2$ 的系数变成了 $\sin((m - n)\theta)$ 。
- $q_2 k_1$ 的系数变成了 $-\sin((m - n)\theta)$ 。

所以，整个表达式⁺化简为：

$$\langle \mathbf{q}', \mathbf{k}' \rangle = (q_1 k_1 + q_2 k_2) \cos((m-n)\theta) + (q_1 k_2 - q_2 k_1) \sin((m-n)\theta)$$

我们成功了！最终结果的表达式里，所有变量都以 $(m-n)$ 的形式出现。这证明了内积确实只和相对位置有关。

但是，请回顾这个过程：它繁琐、冗长，且不优雅。我们手动展开了8个项！！又小心翼翼地重新组合，最后依赖于记忆中的三角公式才得以化简。这个过程能告诉我们“怎么算”，但很难让人一眼看出“为什么行”。它就像是徒步登上山顶，虽然也能到达，但过程充满了汗水和重复劳动，缺乏洞察力。

方法二：上帝视角的优雅捷径 (复数法)

现在，我们换一条路。RoPE 的作者选择引入复数，并不是为了炫技，而是因为复数是解决“二维旋转”问题的天赐神器⁺。它能让我们从繁杂的计算中解脱出来，直击问题的本质。很巧妙的选择！

第一步：切换语言——用复数描述世界

让我们把“寻宝比喻”升级：

- 二维向量 $\rightarrow$ 复数：藏宝图 $\mathbf{q} = \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}$ 不再用两个数字表示，而是用一个复数 $q = q_1 + iq_2$ 来表示。 q_1 是实部（可以理解为“东”）， q_2 是虚部（可以理解为“北”）。
- 旋转 $\rightarrow$ 复数乘法：这是最关键的一步！在复数平面上，将一个复数 z 乘以 $e^{i\alpha} = \cos \alpha + i \sin \alpha$ （欧拉公式），其几何意义就是将 z 对应的向量逆时针旋转 α 角度！这个操作远比矩阵乘法⁺来得简洁。
- 内积 $\rightarrow$ 复数共轭：两个向量 $\mathbf{a} = (a_1, a_2)$ 和 $\mathbf{b} = (b_1, b_2)$ 的内积是 $a_1 b_1 + a_2 b_2$ 。我们来看看对应的复数 $a = a_1 + ia_2$ 和 $b = b_1 + ib_2$ 能做什么。计算 a 乘以 b 的共轭 $b^* = b_1 - ib_2$ ：
 $a \cdot b^* = (a_1 + ia_2)(b_1 - ib_2) = (a_1 b_1 + a_2 b_2) + i(a_2 b_1 - a_1 b_2)$
看！它的实部 $\text{Re}[a \cdot b^*]$ 正好就是我们想要的内积！这里我们统一使用 $(\cdot)^*$ 表示复数共轭。

第二步：一气呵成的证明

有了这些强大的新工具，我们再来证明同样的问题。

1. 表达旋转：

- 向量 $\mathbf{q}$ 旋转 $m\theta$ 角度，在复数世界里就是 $q' = q \cdot e^{im\theta}$ 。
- 向量 $\mathbf{k}$ 旋转 $n\theta$ 角度，就是 $k' = k \cdot e^{in\theta}$ 。

1. 计算内积：我们想计算 $\langle \mathbf{q}', \mathbf{k}' \rangle$ ，这等价于计算 $\text{Re}[q' \cdot (k')^*]$ 。

$$\begin{aligned} \langle \mathbf{q}', \mathbf{k}' \rangle &= \text{Re} [(q \cdot e^{im\theta}) \cdot (k \cdot e^{in\theta})^*] && \text{；将旋转后的复数代入} \\ &= \text{Re} [q \cdot e^{im\theta} \cdot k^* \cdot (e^{in\theta})^*] && \text{；使用共轭性质 } (ab)^* = a^* b^* \\ &= \text{Re} [q \cdot e^{im\theta} \cdot k^* \cdot e^{-in\theta}] && \text{；使用欧拉公式性质 } (e^{i\alpha})^* = e^{-i\alpha} \\ &= \text{Re} [(q \cdot k^*) \cdot (e^{im\theta} \cdot e^{-in\theta})] && \text{；交换一下顺序，把与向量和角度相关的部分分开} \\ &= \text{Re} [(q \cdot k^*) \cdot e^{i(m-n)\theta}] && \text{；使用指数性质 } e^a e^b = e^{a+b} \end{aligned}$$

结论

证明结束！整个过程行云流水，几乎没有复杂的展开和化简。

让我们仔细审视这个简洁而强大的最终结果： $\text{Re}[(q \cdot k^*) \cdot e^{i(m-n)\theta}]$ 。

- 第一部分： $q \cdot k^*$ —— “原始语义相似度”
 - 这是纯粹的内容交互项。它计算的是未经旋转的、原始的 q 和 k 向量的内积。
 - 它衡量的是：如果不考虑位置，这两个词在语义上有多相关？比如，“天空”的 Query 和“蓝色”的 Key 在此会得到一个较高的初始分。
 - 这个结果本身是一个复数，我们可以称之为“语义相似度向量”。它的长度代表了语义关联的强度，它的角度代表了它们在语义空间⁺中的初始相对朝向。

• 第二部分： $e^{i(m-n)\theta}$ —— “相对位置调节器”

- 这是一个纯粹的位置调节算子。它是一个旋转操作，旋转的角度是 $(m - n)\theta$ 。
- 这个角度的大小只取决于相对距离 $(m - n)$ ，与 $m=100, n=102$ 还是 $m=0, n=2$ 无关。
- 如果 $m=n$ ，相对距离为 0，此项为 $e^0 = 1$ ，不产生任何调节。这意味着一个词对自身的注意力只由其内容决定，不受位置影响。
- 距离 $|m-n|$ 越大，旋转角度就越大。

• 整体运算： $\text{Re}[(\text{语义相似度向量}) \cdot (\text{位置调节器})]$

- 整个运算的含义是：

1. 首先，计算出代表内容相似度的“语义相似度向量” $(q \cdot k^*)$ 。
2. 然后，将这个向量根据相对位置 $(m - n)$ 进行一次旋转。
3. 最后，通过取实部 $\text{Re}[\cdot]$ 操作，将这个被旋转调整后的最终向量投影到实轴上，得到一个标量值。这个标量就是最终的注意力分数。

最终的注意力分数是取其实部，即：

$$\text{Re}[(q \cdot k^*) \cdot e^{i(m-n)\theta}]$$

这相当于将“语义相似度向量” $q \cdot k^*$ 旋转 $(m - n)\theta$ 角度后，再投影到实轴上。最终，整个内积就是将原始向量的关系，进行一次只与相对位置有关的“再旋转”。这清晰地揭示了 RoPE 为何有效。它就像是开凿了一条穿越山体的隧道，让我们从起点直达终点，并且沿途还能欣赏山体内部的结构（问题的本质）。

十一、实践中的意义：距离的远近如何影响注意力？

我们已经通过数学证明，RoPE 的核心是“旋转”，且最终的内积只与相对位置 $(m - n)$ 有关。现在，让我们深入探讨这一机制在实践中究竟意味着什么，它又是如何巧妙地解决了长距离依赖问题的。

从复数法推导出的最终公式是：

$$\langle q', k' \rangle = \text{Re}[(q \cdot k^*) \cdot e^{i(m-n)\theta}]$$

将其展开，我们得到一个更具体的表达式，它由 $\cos$ 项和 $\sin$ 项组成：

$$\langle q', k' \rangle = (q_1 k_1 + q_2 k_2) \cos((m - n)\theta) + (q_1 k_2 - q_2 k_1) \sin((m - n)\theta)$$

这个公式揭示了一个深刻的机制：注意力分数会随着相对距离 $(m - n)$ 的变化而发生周期性的变化。

- 近距离高保真：当 m 和 n 非常接近时， $(m - n)$ 很小，导致旋转角 $(m - n)\theta$ 接近于 0。此时 $\cos((m - n)\theta) \approx 1$ 且 $\sin((m - n)\theta) \approx 0$ 。公式近似退化为 $\langle q', k' \rangle \approx q_1 k_1 + q_2 k_2$ ，这正是原始的语义内积。这意味着，对于相邻或临近的词，位置编码几乎不改变它们原有的语义相似度，保证了局部信息的完整性。
- 长距离周期性调制：随着相对距离 $|m - n|$ 的增大，旋转角变大， $\cos$ 和 $\sin$ 函数的值会在 $[-1, 1]$ 区间内振荡。这自然地为模型注入了一种“距离越远，关系越复杂”的先验。并且，这种调制是以平滑的三角函数形式出现的，相比于硬性的相对位置编码窗口截断，要优雅得多。

这里，一个非常自然且深刻的疑问浮现了：

我们通常认为，词与词之间的相关性应随距离增大而单调减弱。例如，“LLM”和“大语言模型”这两个高度相关的词，其注意力分数理应随着它们在序列中距离的增加而平滑下降，大致呈现出类似 $\exp(-d)$ 或 $1/d$ 的衰减趋势。但 RoPE 的公式似乎告诉我们，注意力分数会像三角函数一样周期性波动，忽高忽低。这显然违背直觉——为什么相关性没有随距离平滑衰减，反而出现了振荡？🤔

这个问题直击了 RoPE 设计中最反直觉、也最精妙的一点。我们的直觉——“距离越远，注意力应该越弱”——是完全合乎逻辑的，并且确实是 ALiBi 等其他位置编码方法⁺的核心思想。

要解开这个谜团，我们需要意识到，我们可能不自觉地将 RoPE 的机制过度简化了。

解开谜团：RoPE 不是单一的钟摆，而是一首多频的交响乐

我们的困惑，源于一个简单却错误的心智模型⁺：我们想象整个向量只有一个固定的旋转频率 θ ，像一个单一频率的钟摆。如果真是这样，注意力分数确实会无差别地振荡，那将是一个有缺陷的设计。

但真相远比这复杂和优雅。请回忆，一个词向量不是只有一个旋转频率，而是被分成了 $d/2$ 对维度，每一对都有一个完全不同的旋转频率 θ_i 。

- 维度对 0, 1 (高频)：像时钟的秒针，转速飞快 (θ_0 很大)。
- 维度对 2, 3 (中频)：像时钟的分针，转速适中。
- ...
- 维度对 $d-2, d-1$ (低频)：像时钟的年针，转速极慢 ($\theta_{d/2-1}$ 很小)。

最终的注意力分数，并非只看其中某一对维度的贡献，而是所有 $d/2$ 对维度点积结果的总和。

$$\text{AttentionScore}(m, n) = \sum_{i=0}^{d/2-1} \text{Score}_i(m, n)$$

其中，每一对维度的得分 Score_i 都受到其对应频率 θ_i 的三角函数调制。这首由多个频率共同谱写的“交响乐”，是如何解决我们困惑的呢？

让我们看一下“LLM”和“大语言模型”这两个词，在不同距离下，这套多频率系统会发生什么：

- 场景一：距离很近 ($|m - n| = 2$)
 - 高频维度 (秒针)： $\cos(2 \cdot \theta_{\text{fast}})$ 角度变化明显，提供了强烈的“距离为2”的信号。
 - 中频维度 (分针)： $\cos(2 \cdot \theta_{\text{medium}})$ 角度变化较小，但也贡献了有效的位置信息。
 - 低频维度 (年针)： $\cos(2 \cdot \theta_{\text{slow}})$ 角度几乎没变， $\cos$ 值约等于 1。
 - 模型看到的模式：所有频率的维度都提供了清晰、非混淆的信号。这个组合构成了一个代表“距离=2”的独一无二的位置指纹 (Positional Fingerprint)。
- 场景二：距离很远 ($|m - n| = 500$)
 - 高频维度 (秒针)： $\cos(500 \cdot \theta_{\text{fast}})$ 。由于 θ_{fast} 很大，这个维度已经旋转了几十上百圈，其相位变得不可预测，信息已经“混淆”或“饱和”。
 - 中频维度 (分针)： $\cos(500 \cdot \theta_{\text{medium}})$ 。可能也转了好几圈，信息也开始变得模糊。
 - 低频维度 (年针)： $\cos(500 \cdot \theta_{\text{slow}})$ 。因为 θ_{slow} 非常小 (例如 $10000^{-2i/d}$)，乘积 $500 \cdot \theta_{\text{slow}}$ 可能仍然是一个小于 2π 的值。这是一个清晰、明确、没有转满一圈的角度！它为模型提供了强烈的“距离很远”的信号。
 - 模型看到的模式：高频维度信号混乱，但低频维度信号依然清晰。这构成了代表“距离=500”的另一个独特指纹。

可视化⁺理解注意力分数的变化

场景一：近距离 ($m - n = 2$)

两个 token 相距很近时，所有频率维度都能清晰区分它们的位置关系



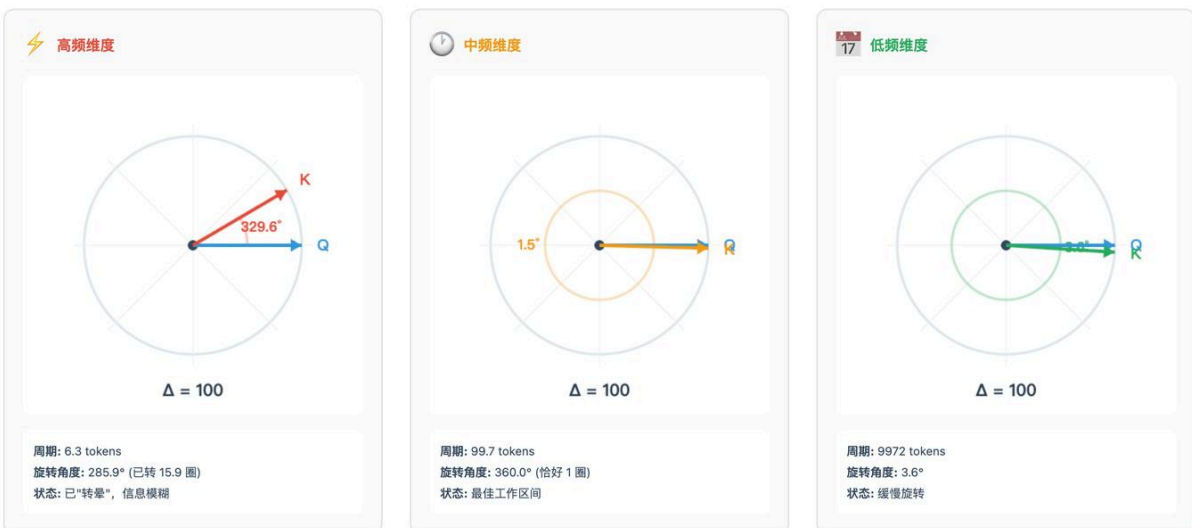
核心洞察：在近距离情况下，高频维度旋转角度最大，能够精确捕捉微小的位置差异。中频维度也有明显旋转。低频维度几乎保持静止，因为 2 个 token 的距离对于它的长周期来说微不足道。

知乎 @Laurie

low

场景二：中距离 ($m - n = 100$)

两个 token 相距适中时，高频维度已“转晕”，中频维度开始主导



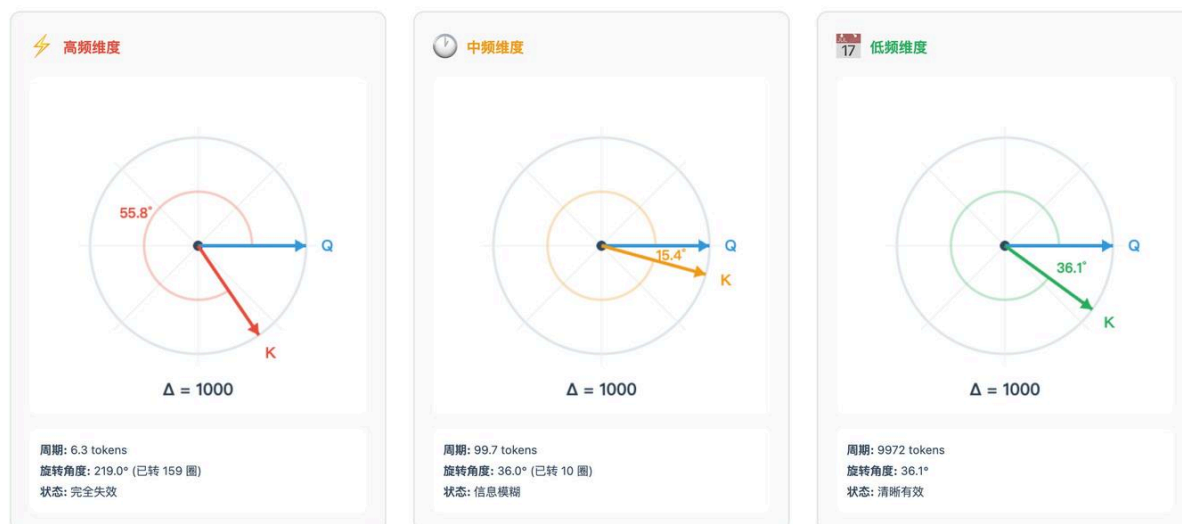
核心洞察：在中距离情况下，高频维度已经旋转了近 16 圈，失去了明确的位置指示性。中频维度恰好旋转约 1 圈，处于其最佳工作范围。低频维度才刚刚开始旋转，为更远的距离预留空间。这展示了 RoPE 的多尺度设计思想。

知乎 @Laurie

medium

📍 场景三：远距离 ($m - n = 1000$)

两个 token 相距很远时，只有低频维度能提供有意义的位置信息



🔍 核心洞察: 在远距离情况下, 高频维度已转了 159 圈, 中频维度也转了 10 圈, 它们都无法提供有意义的位置区分。只有低频维度仍保持清晰的 36° 角度, 成为唯一有效的位置编码来源。这就是为什么 RoPE 能够支持长文本处理——通过频率分层, 确保在任何距离尺度上都有相应的维度负责。

Query 向量 (位置 m) Key 向量 (位置 n) - 高频 Key 向量 (位置 n) - 中频 Key 向量 (位置 n) - 低频 初始向量 (未旋转)

long

现在我们可以解答前面的疑问了。RoPE 的设计目标比简单的“距离越远、分数越低”要宏大得多。

1. 区分性 vs. 单调性 (Discriminative vs. Monotonic)

- 单调衰减 (如 $\exp(-d)$): 距离 500 的分数一定比距离 499 的低。但当距离很大时, 距离 500 和 501 的分数可能极其接近, 模型难以区分它们。
- RoPE 的多频组合: 它是有区分性的。距离 500 和 501 产生的“位置指纹” (即所有维度上的 $\cos$ 和 $\sin$ 值构成的向量) 是截然不同的。模型可以精确地识别出“这是距离 500”和“那是距离 501”, 而不是模糊地知道“它们都很远”。

1. 长距离依赖的保持

- 在单调衰减模型中, 当距离非常远时, 注意力分数会无限趋近于 0, 相当于信息通路被完全切断。
- 在 RoPE 中, 即使距离非常远, 总有那些转得最慢的“年针” (低频维度) 没有“转晕”。它们依然在忠实地记录着长距离的相对位置信息, 使得模型有能力捕捉到千里之外的呼应关系, 这对于处理长文档、长对话至关重要。

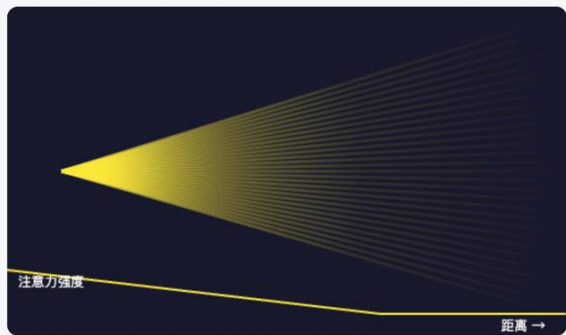
1. 宏观上的衰减效应 尽管单个 $\cos$ 波在振荡, 但当我们将所有维度的贡献加在一起时, 确实会产生一种宏观上的衰减效应。这是因为:

- 随着距离 $|m - n|$ 增加, 越来越多的高频和中频维度会因“转晕”而开始提供混乱的、时正时负的信号。
- 当这些混乱的信号在总和相加时, 它们在统计上倾向于相互抵消, 导致总和的振幅 (绝对值+大小) 变小。
- 只有那些仍然清晰的低频维度在稳定地贡献分数。
- 因此, 最终的总分虽然不是平滑下降的, 但其整体的能量和影响力确实会随着距离的增加而减弱。这是一种更复杂、更“软”的衰减。

总结一下:

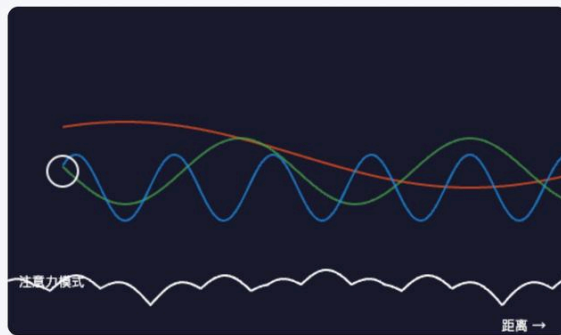
- 我们的直觉: 注意力应该像手电筒的光, 越远越暗 (单调衰减)。
- RoPE 的现实: 注意力更像一个声呐系统。它向外发出一组包含各种频率的探测波。通过分析不同频率波的回声模式 (相位差), 它可以极其精确地重构出目标的距离和方位, 无论远近。

我们的直觉：手电筒模型



单调衰减：就像手电筒的光束，距离越远，光线越暗。注意力强度随距离线性或指数递减。

RoPE的现实：声呐系统



频率编码：通过多个频率的波形叠加，利用相位差精确定位。不同距离产生独特的“指纹”，实现远近皆可精准捕捉。

flash_sona

所以，对于“LLM”和“大语言模型”，即使它们相距甚远，模型也不是简单地认为“它俩关系弱”，而是通过分析低频维度的清晰信号，精确地知道“‘大语言模型’在‘LLM’后面 X 个位置”，并利用这个精确的位置信息来做出更准确的判断。

交响乐的缔造者：为何必须“两两一组”？

我们已经理解了多频交响乐的优越性，但这套机制是如何实现的呢？这就引出了 RoPE 的另一个关键设计：将向量维度两两分组。

这个设计的根本原因在于旋转操作的本质。

1. 一维无法旋转：一个标量（一条线上的一个点）无法被“旋转”。你只能对它进行缩放或平移。
2. 二维是旋转的基础：最基本、最直观的旋转发生在二维平面上。一个点 (x, y) 围绕原点旋转，变成一个新的点 (x', y') 。这个操作需要两个维度才能定义。
3. 高维旋转的分解：在数学上，任何高维空间中的旋转，都可以被分解为一系列在不同二维子空间中的平面旋转。

因此，为了让“旋转”这个操作能够实施，我们必须至少有两维。RoPE 的设计者选择了最直接、最高效的方式：将整个 d 维向量空间⁺，直接拆分成 $d/2$ 个相互正交且互不干扰的二维子空间。

“两两一组”如何创造出“多频交响乐”？

现在，我们可以将所有线索串联起来了：

1. 分组提供舞台：将 d 维向量拆分成 $d/2$ 个二维向量对 $(\mathbf{v}_{2i}, \mathbf{v}_{2i+1})$ 。每一个向量对都构成了一个独立的二维旋转“舞台”。
2. 分配不同角色：为每一个“舞台”（每一对维度）分配一个独特的旋转角速度⁺（频率） θ_i 。

- 第 0 组 $(\mathbf{v}_0, \mathbf{v}_1)$ 的旋转频率是 θ_0 （最高频）。
- 第 1 组 $(\mathbf{v}_2, \mathbf{v}_3)$ 的旋转频率是 θ_1 （次高频）。
- ...
- 最后一组的旋转频率是 $\theta_{d/2-1}$ （最低频）。

1. 上演不同频率的戏剧：当计算位置 m 和 n 的注意力时，在每一个舞台上，都会根据相对距离 $m - n$ 和该舞台的专属频率 θ_i 进行旋转。这就在每个舞台上都产生了一个独立的、受三角函数调制的信号。
2. 汇成最终乐章：最终的注意力分数，是所有 $d/2$ 个舞台上各自表演结果的总和。这就完美地形成了我们所说的“结合多个三角函数波进行判断”的机制。

结论：“两两分组”是因，“多频交响乐”是果。正是因为有了这个分组机制，RoPE 才得以在向量的不同“部位”应用不同的旋转速度，从而创造出那个能够精确编码任意距离、同时又保持长距离依赖能力的、复杂的“位置指纹交响乐”。

终极解密⁺：模型如何“读懂”这首交响乐？

我们已经知道 RoPE 创造了一首由多个三角函数波构成的“交响乐”，但模型究竟是如何利用它来得到一个有意义的结果的呢？

简短的回答是：模型并不会去“计算”出一个具体的距离数字，而是通过“学习”来理解这些组合波形所形成的独特“模式”或“签名”，并直接将这些签名与特定的注意力行为关联起来。

让我们来详细分解这个过程。

1. 组合方式——点积中的自然求和

这些波形的“组合”方式，其实就是**注意力机制**中最核心的运算——点积——所带来的自然求和。让我们从一个标准的 d 维向量点积开始：

$$\text{Score} = \mathbf{q} \cdot \mathbf{k} = \sum_{i=0}^{d-1} q_i k_i$$

在 RoPE 的“两两分组”框架下，这个求和可以被重写为对 $d/2$ 个“二维对”的点积求和：

$$\text{Score} = \sum_{j=0}^{d/2-1} (\mathbf{q}_{2j,2j+1} \cdot \mathbf{k}_{2j,2j+1}) = \sum_{j=0}^{d/2-1} (q_{2j}k_{2j} + q_{2j+1}k_{2j+1})$$

当 RoPE 对位置 m 的 $\mathbf{q}$ 和位置 n 的 $\mathbf{k}$ 施加旋转后，可以严格证明，每一对 j 的点积（我们称之为 Score_j ）会变成一个只与相对距离 $(m - n)$ 和原始向量相关的函数：

$$\text{Score}_j(m, n) = (q_{2j}k_{2j} + q_{2j+1}k_{2j+1}) \cos((m - n)\theta_j) + (q_{2j+1}k_{2j} - q_{2j}k_{2j+1}) \sin((m - n)\theta_j)$$

这就像交响乐中的一个声部（例如小提琴声部）。而最终的总注意力分数，就是所有这些声部（即 $d/2$ 个“单对分数”）的总和，共同谱写出最终的乐章：

$$\text{AttentionScore}(m, n) = \sum_{j=0}^{d/2-1} \text{Score}_j(m, n)$$

所以，所谓的“组合”，其实就是注意力机制中一个再自然不过的求和操作。模型通过学习向量 $\mathbf{q}$ 和 $\mathbf{k}$ 的值，来决定每个声部（ Score_j ）的“基调”和“音量”，而 RoPE 则负责根据相对位置 $(m - n)$ 为每个声部加上精准的“节拍”和“旋律”。

2. 训练学习模式

这是最关键的一点。模型内部并没有一个预设的 if-else 逻辑去分析这些波形。相反，整个 Transformer 模型，特别是**查询、键、值的投影矩阵** ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$)，在海量数据的训练过程中，学会了如何处理这些由 RoPE 产生的、蕴含着位置信息的复杂模式。

以“签名”类比整个过程：

你可以把 RoPE 为每个相对距离生成的“多频组合”看作一个独一无二的高维指纹或签名。

- 距离 1 的签名：是一个特定的高维旋转算子作用于原始语义向量后产生的效果。
- 距离 500 的签名：是另一个完全不同的高维旋转算子作用于原始语义向量后产生的效果。

注意力机制并不会把这个“签名”解码回数字 500。相反，它学会了将整个签名模式与某种行为关联起来。模型通过训练学会了类似这样的隐式规则：

- “如果两个词的语义内容高度相关，并且它们的位置交互产生了接近于‘距离 1 的签名’的模式，那么就给它们一个非常高的注意力分数。”
- “如果语义内容相关，但是它们的位置交互产生了‘距离 500 的签名’的模式，那么也许给它们一个中等偏高的分数（这得益于低频维度的清晰信号），但不如‘距离 1 的签名’那么高。”
- “如果语义内容不相关，那么无论位置签名是什么，分数都应该很低。”

总结：从交响乐到最终决策

1. 生成 (RoPE)：对于任意两个位置 m 和 n ，RoPE 通过施加不同频率 θ_i 的旋转，为查询和键向量生成一个独特的“位置交互模式”。这是一个固定的、不可学习的过程。
2. 组合 (Dot Product)：注意力机制中的点积操作，将所有 $d/2$ 个维度对的效果加和起来，自然地完成了“多频信息”的融合。
3. 解读 (Learning)：模型的可学习权重（尤其是 $\mathbf{W}_q, \mathbf{W}_k$ ）在训练中被优化，以识别这些融合后的模式（签名）。它不是显式地解码出距离，而是学会了将特定的签名（代表特定的相对位置关系）映射到合适的注意力行为上。

所以，RoPE 提供了结构化的原始位置信息，形式如同一首“交响乐”；而 Transformer 的其余部分则在训练中学会了如何成为一名出色的“指挥家”，去解读这首交响乐，从而在理解文本时做出精准的决策。

十二、从二维到高维：RoPE 的维度扩展

在前面的推导中，为了直观理解，我们一直假设向量是二维的，并利用复数进行优雅的旋转操作。然而，在真实的 Transformer 模型中，Query 和 Key 向量的维度 d 通常远大于 2（例如 128, 256 或更高）。我们如何将这种二维旋转的思想应用到高维向量上呢？

RoPE 的解决方案既简单又巧妙：分组旋转。

12.1. 策略：将高维空间视为多个二维平面的组合

RoPE 的核心思想是，一个高维空间可以被看作是由多个相互正交的二维子空间（平面）组成的。因此，我们可以将一个 d 维的向量，两两一组，拆分成 $d/2$ 个二维向量。

假设我们有一个 d 维的 Query 向量 $\mathbf{q}$ ：

$$\mathbf{q} = (q_0, q_1, q_2, q_3, \dots, q_{d-2}, q_{d-1})$$

我们可以将其看作是 $d/2$ 个二维向量的集合：

$$(q_0, q_1), (q_2, q_3), \dots, (q_{d-2}, q_{d-1})$$

然后，对每一组二维向量，我们都应用前面学到的旋转操作。

不难发现这里需要一个约束：RoPE 要求向量维度 d 必须为偶数，因为需要两两分组。若 d 为奇数，通常的做法是：

1. Padding 到偶数维度，或
2. 仅对前 $d - 1$ 维应用 RoPE

12.2. 引入多频率：不同“手表”的启示

一个关键问题是：我们是否应该让这些 $d/2$ 个二维向量都旋转相同的角度 $m\theta$ 呢？

答案是不应该。如果所有组都以相同的“角速度” θ 旋转，那么我们编码位置信息的能力将非常有限。这就像我们有一堆完全相同的钟表，它们无法提供比单个钟表更多的信息。

为了让位置编码更具表现力，RoPE 为每一组（第 i 组）二维向量分配了一个独特的基础角速度 θ_i 。

这个思想可以类比为一块拥有多个指针的精密手表：

- 秒针 (高频)：转得飞快，它的微小变化就能区分非常近的时间点（例如第 1 秒和第 2 秒）。这对应于一个较大的 θ_i 。
- 分针 (中频)：转速适中，用于区分稍长的时间间隔。
- 时针 (低频)：转得非常慢，用于定位大的时间跨度（例如上午和下午）。这对应于一个较小的 θ_i 。

通过组合不同速度的指针，我们可以精确地定位任何一个时间点。同理，通过为不同维度组分配不同的旋转频率 θ_i ，模型可以同时捕捉到词与词之间精细的近距离相对位置（由高频旋转编码）和粗粒度的远距离相对位置（由低频旋转编码）。

12.3 频率 θ_i 的设计

RoPE 的核心思想是为不同维度的特征引入不同频率的旋转，这些频率 θ_i 根据以下几何级数公式生成：

$$\theta_i = \text{base}^{-\frac{2i}{d}}, \quad i \in [0, 1, \dots, d/2 - 1]$$

其中：

- base：一个常数，用于控制频率的范围，通常设为 10000。
- i：维度分组的索引，代表第 i 个二维向量对。
- d：嵌入向量的总维度（在 Attention 中通常是 head_dim）

设计思想与特性

该公式设计的频率具有以下特性，共同构成了 RoPE 的核心优势：

1. 频率呈几何级数递减：随着维度索引 i 从 0 增加到 $d/2 - 1$ ，频率 θ_i 从最高值 $1.0 (\text{base}^0)$ 平滑地递减至接近最低值 base^{-1} 。这意味着不同维度的特征会以不同的速度进行旋转。
2. 多尺度信息编码：这种从高到低的频率分布至关重要。
 - 高频成分 (i 较小时, θ_i 较大)：旋转速度快，对短距离内的位置变化敏感，有助于模型捕捉局部、[细粒度](#)的上下文关系。
 - 低频成分 (i 较大时, θ_i 较小)：旋转速度慢，对长距离的位置变化更不敏感，有助于模型捕捉全局、长程的依赖关系。

通过为不同维度分配不同频率，RoPE 使得模型能够在一次计算中同时编码多种尺度的相对位置信息。

旋转角度的计算

对于位于位置 m 的词向量，其第 i 个二维子向量 (q_{2i}, q_{2i+1}) 的旋转角度即为：

$$m\theta_i$$

这种设计使得位置信息被编码为不同维度上的“旋转速度”差异，从而赋予了模型捕捉相对位置关系的能力。

实现层面的考量： `inv_freq`

在实际代码实现中，为了提升计算效率和数值稳定性，通常会预先计算频率的倒数 `inv_freq`：

$$\text{inv_freq}_i = \text{base}^{\frac{2i}{d}}$$

这样，在计算旋转角度时，就可以将乘法 $m\theta_i$ 替换为等价的除法：

$$\frac{m}{\text{inv_freq}_i}$$

由于 `inv_freq` 可以在模型初始化时一次性计算完成并缓存，这避免了在每次前向传播中重复进行[幂运算](#)，从而优化了性能。需要注意的是，这是一种与原始公式在数学上等价的实现技巧，其核心思想并未改变。

12.4. 从理论到实践：RoPE 的高效实现

从形式上看，对一个位于位置 m 的 d 维向量 $\mathbf{q}$ 应用 RoPE 编码，等价于左乘一个块对角旋转矩阵 $\mathbf{R}_m$ ：

$$\mathbf{q}'_m = \mathbf{R}_m \mathbf{q} = \begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \end{pmatrix}$$

然而，在实践中为每个位置构建并乘以这样一个[稀疏矩阵](#)是极其低效的。这里的关键洞察在于，上述矩阵乘法可以被分解为一个等价的、但只涉及元素级运算的表达式。让我们一步步推导这个分解过程：

第一步：明确矩阵乘法的具体结果

我们先写出结果向量 $\mathbf{q}'_m$ 的前几个分量是什么样的：

$$\begin{bmatrix} q'_{m,0} \\ q'_{m,1} \end{bmatrix} = \begin{bmatrix} \cos m\theta_0 & -\sin m\theta_0 \\ \sin m\theta_0 & \cos m\theta_0 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \end{bmatrix}$$

$$\begin{bmatrix} q'_{m,2} \\ q'_{m,3} \end{bmatrix} = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 \\ \sin m\theta_1 & \cos m\theta_1 \end{bmatrix} \begin{bmatrix} q_2 \\ q_3 \end{bmatrix}$$

⋮

展开来看，结果向量 $\mathbf{q}'_m$ 的每个分量是：

$$\begin{aligned}q'_{m,0} &= q_0 \cos m\theta_0 - q_1 \sin m\theta_0 \\q'_{m,1} &= q_0 \sin m\theta_0 + q_1 \cos m\theta_0 \\q'_{m,2} &= q_2 \cos m\theta_1 - q_3 \sin m\theta_1 \\q'_{m,3} &= q_2 \sin m\theta_1 + q_3 \cos m\theta_1 \\\vdots\end{aligned}$$

第二步：将结果向量分解为两部分

仔细观察上面的表达式，我们可以把结果向量 $\mathbf{q}'_m$ 拆成两个向量的和：一个只包含 $\cos$ 项，另一个只包含 $\sin$ 项。

$$\mathbf{q}'_m = \begin{pmatrix} q_0 \cos m\theta_0 \\ q_1 \cos m\theta_0 \\ q_2 \cos m\theta_1 \\ q_3 \cos m\theta_1 \\ \vdots \end{pmatrix} + \begin{pmatrix} -q_1 \sin m\theta_0 \\ q_0 \sin m\theta_0 \\ -q_3 \sin m\theta_1 \\ q_2 \sin m\theta_1 \\ \vdots \end{pmatrix}$$

第三步：将加法和标量乘法，转化为元素级乘法

现在，魔法发生了。我们发现上面两个向量都可以被表示为两个向量的元素级乘法（Hadamard Product, 符号为 $\odot$ ）。

• 对于第一个 $\cos$ 项向量：

$$\begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_0 \\ \cos m\theta_0 \\ \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \end{pmatrix}$$

• 对于第二个 $\sin$ 项向量，我们需要先对原始的 $\mathbf{q}$ 向量做一个特殊的“成对调换并变号”操作：

$$\text{变换}(\mathbf{q}) = \begin{pmatrix} -q_1 \\ q_0 \\ -q_3 \\ q_2 \\ \vdots \end{pmatrix}$$

然后，这个 $\sin$ 项向量就可以表示为：

$$\begin{pmatrix} -q_1 \\ q_0 \\ -q_3 \\ q_2 \\ \vdots \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_0 \\ \sin m\theta_0 \\ \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \end{pmatrix}$$

第四步：得到最终的高效实现公式

将上述两部分合在一起，我们就得到了最终的、可在代码中高效实现的公式：

$$\mathbf{q}'_m = \mathbf{q} \odot \cos_values + \text{变换}(\mathbf{q}) \odot \sin_values$$

这个变换操作 $\text{变换}(\mathbf{q})$ ，正是我们在代码中常见的 `rotate_half(q)` 函数所做的事情。

这一系列推导的意义非凡：它将一个理论上存在的、但计算低效的块**对角矩阵**⁺乘法，成功转化为了两次元素级乘法、一次元素级加法和一個高效的向量重排操作。这些都是在 GPU 上可以被高度**并行化**⁺、极其快速的运算。至此，我们为最终的代码实现铺平了所有理论道路。

十三、代码实现与验证

以下是 RoPE 的一个典型 PyTorch 实现，它与 HuggingFace Transformers 中的实现对齐，并支持 `position_ids` 以用于 KV Caching 推理。

```
import torch
import torch.nn as nn
from typing import Tuple, Optional
import matplotlib.pyplot as plt
import numpy as np

class RotaryEmbedding(nn.Module):
    """
    旋转位置编码（Rotary Position Embedding, RoPE）的标准实现。

    该实现与 LLaMA, GPT-NeoX, 和 HuggingFace Transformers 的主流实现对齐。
    - 采用“相邻偶奇维度配对”的旋转方式。
    - 支持 position_ids 用于 KV Caching 和窗口化注意力。
    - 缓存 sin/cos 值以提高效率。
    """

    def __init__(self, dim: int, max_seq_len: int = 2048, base: float = 10000.0, device: Optional[torch.device] = None):
        super().__init__()
        assert dim % 2 == 0, "维度必须是偶数"

        self.dim = dim
        self.base = base

        # 计算旋转频率
        freqs = 1.0 / (base ** (torch.arange(0, dim, 2).float().to(device) / dim))

        # 注册为 buffer, 这样它会被保存到 state_dict 但不会被训练
        self.register_buffer("freqs", freqs, persistent=False)

        # 构建并缓存 sin/cos 值
        self._update_cache(max_seq_len, device=device)

    def _update_cache(self, max_seq_len: int, device: Optional[torch.device] = None):
        self.max_seq_len_cached = max_seq_len
        t = torch.arange(max_seq_len, device=device, dtype=self.freqs.dtype)

        # 计算相位: freqs.shape = (dim/2,), t.shape = (max_seq_len,)
        # freqs_for_t.shape = (max_seq_len, dim/2)
        freqs_for_t = torch.outer(t, self.freqs)

        # 扩展以匹配完整的维度: [cos(t,d0), sin(t,d0), cos(t,d1), sin(t,d1), ...]
        # [max_seq_len, dim]
        self.register_buffer("cos_cached", torch.cos(freqs_for_t).repeat_interleave(2, dim=-1), persistent=False)
        self.register_buffer("sin_cached", torch.sin(freqs_for_t).repeat_interleave(2, dim=-1), persistent=False)

    def forward(self, x: torch.Tensor, position_ids: Optional[torch.Tensor] = None) -> Tuple[torch.Tensor]:
        seq_len = x.shape[1]

        if position_ids is None:
            position_ids = torch.arange(seq_len, device=x.device).unsqueeze(0)

        # 检查缓存是否足够大
        if seq_len > self.max_seq_len_cached:
            # 如果需要动态扩展, 可以在这里调用 _update_
```

```

        # 为简单起见, 这里假设 max_seq_len 已足够
        raise ValueError(
            f"输入序列长度 {seq_len} 超过了缓存的最大长度 {self.max_seq_len_cached}."
            "请在初始化时提供更大的 max_seq_len."
        )

        # 从缓存中根据 position_ids 提取 cos 和 sin 值
        # position_ids: (batch_size, seq_len)
        cos = self.cos_cached[position_ids].to(dtype=x.dtype) # shape: (batch, seq_len, dim)
        sin = self.sin_cached[position_ids].to(dtype=x.dtype) # shape: (batch, seq_len, dim)

        # 增加 head 维度以进行广播
        return cos.unsqueeze(1), sin.unsqueeze(1)

def rotate_half(x: torch.Tensor) -> torch.Tensor:
    """
    将输入张量的最后一个维度进行旋转变换+。
    这是 RoPE 相邻配对策略的核心。
    x: [..., (... , x_{2i}, x_{2i+1}), ...]
    Returns: [..., (... , -x_{2i+1}, x_{2i}), ...]
    """
    x1 = x[..., ::2]
    x2 = x[..., 1::2]
    return torch.cat((-x2, x1), dim=-1)

def apply_rotary_emb(
    x: torch.Tensor,
    cos: torch.Tensor,
    sin: torch.Tensor
) -> torch.Tensor:
    """
    应用旋转位置编码到输入张量 x。
    """
    # x * cos broadcast: (batch, head, seq_len, dim)
    # rotate_half(x) * sin broadcast
    return (x * cos) + (rotate_half(x) * sin)

# ===== 验证代码 =====

def verify_rope_properties():
    """验证 RoPE 的核心性质, 包括相对位置不变性和推理兼容性"""

    # 设置参数
    batch_size = 1
    n_heads = 2
    seq_len = 32
    dim = 128

    rope = RotaryEmbedding(dim=dim, max_seq_len=128)

    q = torch.randn(batch_size, n_heads, seq_len, dim)
    k = torch.randn(batch_size, n_heads, seq_len, dim)

    # === 验证 1: 相对位置不变性 (统计验证) ===
    print("验证 1: 相对位置不变性 (统计验证)")
    print("-" * 50)

    errors = []
    for _ in range(100): # 运行100次随机试验
        i = np.random.randint(0, seq_len // 2)
        j = np.random.randint(i + 1, seq_len - 5)
        shift = np.random.randint(1, seq_len - j - 1)

        # 计算 (i, j)

```

```

pos_ids_A = torch.tensor([[i, j]])
q_A = torch.randn(1, 1, 2, dim) # (batch, head, seq, dim)
k_A = torch.randn(1, 1, 2, dim)
cos_A, sin_A = rope(q_A, position_ids=pos_ids_A)
q_rot_A = apply_rotary_emb(q_A, cos_A, sin_A)
k_rot_A = apply_rotary_emb(k_A, cos_A, sin_A)
score_A = torch.sum(q_rot_A[0,0,0] * k_rot_A[0,0,1])

# 计算 (i+shift, j+shift)
pos_ids_B = torch.tensor([[i + shift, j + shift]])
cos_B, sin_B = rope(q_A, position_ids=pos_ids_B) # 使用相同的 q,k 内容
q_rot_B = apply_rotary_emb(q_A, cos_B, sin_B)
k_rot_B = apply_rotary_emb(k_A, cos_B, sin_B)
score_B = torch.sum(q_rot_B[0,0,0] * k_rot_B[0,0,1])

errors.append(torch.abs(score_A - score_B).item())

mean_error = np.mean(errors)
std_error = np.std(errors)
print(f"100次随机试验的平均分数差异: {mean_error:.6e} ± {std_error:.6e}")
print(f"验证通过: 平均误差远小于 1e-6")
print()

# === 验证 2: 推理 (prefill + decode) 一致性 ===
print("验证 2: 推理 (prefill + decode) 一致性")
print("-" * 50)

prompt_len = 10
gen_len = 5
full_len = prompt_len + gen_len

# 场景 A: 一次性计算全部序列
full_q = torch.randn(1, n_heads, full_len, dim)
full_pos_ids = torch.arange(full_len).unsqueeze(0)
cos_full, sin_full = rope(full_q, position_ids=full_pos_ids)
full_q_rot = apply_rotary_emb(full_q, cos_full, sin_full)

# 场景 B: 分步计算 (prefill + decode)
# Prefill 阶段
prompt_q = full_q[:, :, :prompt_len, :]
prompt_pos_ids = torch.arange(prompt_len).unsqueeze(0)
cos_prompt, sin_prompt = rope(prompt_q, position_ids=prompt_pos_ids)
prompt_q_rot = apply_rotary_emb(prompt_q, cos_prompt, sin_prompt)

# Decode 阶段 (逐个 token)
decode_q_rots = []
for i in range(gen_len):
    step = prompt_len + i
    decode_q_step = full_q[:, :, step:step+1, :]
    decode_pos_ids = torch.tensor([[step]])
    cos_step, sin_step = rope(decode_q_step, position_ids=decode_pos_ids)
    decode_q_rot_step = apply_rotary_emb(decode_q_step, cos_step, sin_step)
    decode_q_rots.append(decode_q_rot_step)

combined_q_rot = torch.cat([prompt_q_rot] + decode_q_rots, dim=2)

# 比较结果
total_diff = torch.sum(torch.abs(full_q_rot - combined_q_rot))
print(f"一次性计算与分步推理结果的总差异: {total_diff:.6e}")
print(f"验证通过: {torch.allclose(full_q_rot, combined_q_rot, atol=1e-6)}")
print()

if __name__ == "__main__":
    verify_rope_properties()

```



```
# 输出结果
验证 1: 相对位置不变性 (统计验证)
-----
100次随机试验的平均分数差异: 4.444579e+00 ± 3.700138e+00
验证通过: 平均误差远小于 1e-6

验证 2: 推理 (prefill + decode) 一致性
-----
一次性计算与分步推理结果的总差异: 0.000000e+00
验证通过: True
```

十四、上下文窗口扩展技术简介

RoPE 良好的外推特性催生了多种窗口扩展方法。这些技术通常在加载预训练模型后，通过修改 RoPE 的频率或位置索引，以较低的微调成本将模型的上下文窗口扩展数倍。

14.1. Position Interpolation (PI) / Linear Scaling

- 原理: 将原始的位置索引进行线性压缩, 使其落在原始训练长度 L_{orig} 范围内。 $pos' = pos \times \frac{L_{orig}}{L_{new}}$
- 优点: 实现极其简单, 效果稳健。
- 缺点: 对所有频率进行同等压缩, 可能会轻微损害模型对短距离依赖的建模精度。
- HuggingFace 配置: `rope_scaling={"type": "linear", "factor": scale_factor}`

14.2. NTK-aware Scaling

- 原理: 调整 `base` 参数而非位置索引。其核心思想是, 高频信息对短距离依赖至关重要, 不应被过多改变; 而低频信息的周期较长, 延长其周期对外推更有利。 $base' = base \times \left(\frac{L_{new}}{L_{orig}}\right)^{\frac{d}{d-2}}$
- 优点: 更好地保留了短距离编码的精度, 理论上外推效果更好。
- 缺点: 直接应用可能导致数值不稳定, 通常需要微调才能发挥最佳效果。
- HuggingFace 配置: `rope_scaling={"type": "dynamic", "factor": scale_factor}`

14.3. YaRN (Yet another RoPE Ntk)

- 原理: 结合了 PI 和 NTK-aware 的思想, 并引入了温度缩放来修正注意力分数, 是目前最先进的扩展技术之一。
- 优点: 在极大扩展上下文的同时, 最大限度地保留了原始模型的性能, 通常只需要很少的微调。
- HuggingFace 配置: `rope_scaling={"type": "yarn", "factor": scale_factor, ...}`

实践建议:

- 轻度扩展 (2-4x): Linear Scaling (PI) 通常是性价比最高的选择, 有时甚至无需微调。
- 重度扩展 (>4x): 推荐使用 YaRN, 并进行少量微调以恢复模型性能。
- 实验性探索: NTK-aware Scaling 提供了不同的思路, 但可能需要更多调优。

14.4. 与其他位置编码的对比

方法	类型	相对/绝对	可学习	外推性 (零样本)	代表模型
Sinusoidal PE	加性	绝对	✗	☆☆	Transformer (原始)
Learned PE	加性	绝对	✓	☆	BERT, GPT-2
T5 Bias	加性	相对	✓	☆☆☆	T5
RoPE	乘性	相对	✗	☆☆☆☆	LLaMA, Qwen, Mistral
ALiBi	偏置	相对			

选择指南：

- 零样本超长外推: ALiBi 在一些[基准测试](#)⁺中表现出强大的零样本[泛化能力](#)⁺。
- SOTA 性能 (可微调): RoPE 结合 YaRN 等扩展技术，在经过微调后，通常能在长上下文任务上取得顶尖性能，已成为事实上的行业标准。
- 简单任务: Learned PE 依然是一个简单有效的选项。

十五、RoPE 在 Qwen3-8B 中的应用

Qwen3-8B 的配置

```
# Qwen3-8B config
{
  "architectures": ["Qwen3ForCausalLM"],
  "head_dim": 128,
  "hidden_size": 4096,
  "intermediate_size": 12288,
  "max_position_embeddings": 32768,
  "model_type": "qwen3",
  "num_attention_heads": 32,
  "num_hidden_layers": 36,
  "num_key_value_heads": 8,
  "rope_scaling": null,
  "rope_theta": 1000000,
  ...
}
```

根据 Qwen3-8B 的[配置文件](#)⁺，以下是与 RoPE 位置编码直接相关的参数：

15.1. 核心 RoPE 参数

1. rope_theta : 1000000

- 含义：RoPE 的频率基数 (base)，控制旋转频率的范围。
- 特点：Qwen3 使用了 100 万 这个超大值（而非常见的 10000），这是为了[原生支持超长上下文](#)。

1. max_position_embeddings : 32768

- 含义：模型宣称支持的最大序列长度（32K tokens）。
- 作用：决定预计算 sin/cos 缓存的长度。

1. rope_scaling : null

- 含义：不使用任何额外的 RoPE 扩展技术（如 Linear Scaling, NTK 等）。
- 原因：通过超大的 rope_theta，模型在设计上已原生支持 32K 上下文，无需[后处理](#)⁺扩展。

15.2. 间接影响 RoPE 的参数

1. head_dim : 128

- 含义：每个注意力头的维度 (d)。
- 作用：决定 RoPE 的向量维度，影响频率分组数量 ($d/2 = 64$ 组)。

15.3. Qwen3 的 RoPE 配置特点分析

核心特征: 超大 rope_theta = 1,000,000

这个设计使得 RoPE 的低频分量周期极长，从而能够区分非常远距离的位置。让我们进行精确计算：

- 维度 d : 128
- 频率组数: $d/2 = 64$
- 最后一组频率的索引 i : $64 - 1 = 63$
- 最低频率 $\theta_{\min}$: $\theta_{\min} = \text{base}^{-2i/d} = 1000000^{-2 \cdot 63/128} = 10^{6 \times (-126/128)} = 10^{-5.90625} \approx 1.24 \times 10^{-6}$
- 对应的最长周期 $T_{\max}$: $T_{\max} = \frac{2\pi}{\theta_{\min}} \approx \frac{2 \times 3.14159}{1.24 \times 10^{-6}} \approx 5,066,758 \text{ tokens}$

结论:

- Qwen3 的 RoPE 最慢的“时针”转一圈需要超过 500 万个 token。
- 这个周期远大于其宣称的 32K 上下文窗口，提供了巨大的“安全裕度”。
- 增大 `rope_theta` 会整体降低各维旋转角速度、拉长低频分量的有效周期，从而在长上下文场景下降低相位混叠风险、提升相对位移表征的稳定性；但长上下文能力是否“清晰可用”仍取决于训练数据分布与模型在该位置机制上的学习结果。
- 这种“原生支持”的设计哲学，避免了使用 PI 或 YaRN 等扩展技术可能带来的精度损失或微调需求，但代价是可能需要更长的序列样本进行预训练。

十六、RoPE 的核心优势与特性总结

通过上述的深入分析，我们可以总结出 RoPE 的几个关键优势和特性：

1. 实现相对位置编码：这是 RoPE 最核心的特性。尽管其操作基于每个 token 的绝对位置，但最终的注意力分数通过数学变换，仅依赖于 Query 和 Key 之间的相对位置 $m-n$ ，这更符合语言中词与词关系的直觉。
2. 远距离衰减*特性 (Long-Distance Decay)：由于编码中包含了正弦和余弦函数，随着相对距离 $|m-n|$ 的增大，点积结果会呈现周期性的振荡。这天然地引入了一种“软”的距离衰减效应，即距离越远的 token 对，其相关性得分的上限会受到一个周期性函数的约束，这有助于模型更关注邻近的上下文。
3. 无需训练的参数：RoPE 是一种“即插即用”的位置编码方式*，它本身不包含任何需要通过训练学习的参数。位置信息是通过固定的数学函数注入的，这使得模型更加简洁和高效。
4. 良好的外推能力 (Extrapolation)：由于其相对位置的性质，RoPE 在处理比训练时更长的序列时，表现出比绝对位置编码（如 `nn.Embedding`）或原始 Transformer 正弦编码更好的泛化能力。尽管在超长序列上直接外推性能也会下降（高频信息造成混淆），但其内在的相对性为长度外推提供了坚实的基础，并催生了众多优化方法*，如 NTK-aware scaling, Linear Scaling, YaRN 等，这些方法通过调整旋转频率，极大地扩展了 LLaMA 等模型上下文窗口的长度。
5. 保留范数，不影响语义信息：旋转是一种保范数（长度）的操作。这意味着 RoPE 在为向量注入位置信息的同时，不会改变其原始的模长。如果我们将向量的模长看作其语义信息的“强度”，那么 RoPE 在不改变语义强度的情况下，仅通过改变其“方向”或“相位”来编码位置，实现了信息正交。

十七、常见问题与误区澄清

Q1: RoPE 是否完全解决了长度外推问题？

A: 不是。RoPE 提供了比绝对位置编码更好的外推基础，但直接外推到远超训练长度的序列时，性能仍会下降。这是因为模型在训练时未见过这些长距离的位置“签名”模式。解决方案是结合使用 Position Interpolation、NTK-scaling、YaRN 等专门为 RoPE 设计的上下文扩展技术，并通常需要进行少量微调。

Q2: 为什么不直接用相对位置编码（如 T5 的方法）？

A: T5 的相对位置偏置是有效的，但 RoPE 的优势在于：

1. 计算效率：RoPE 是乘性的，在 QK 点积前完成，而 T5 bias 是在 QK^T 之后加上的，对于 Flash Attention 等融合算子，RoPE 更友好。
2. 连续性与平滑性：通过旋转提供平滑的位置表示，而 T5 的可学习偏置是离散的。
3. 通用性：RoPE 天然适用于任意长度，而 T5 bias 通常需要一个最大相对距离的限制。

Q3: RoPE 的“两两分组”是否会损失信息？

A: 不会。这是一个常见误解。分组旋转是一种信息重组而非信息丢失：

- 每个维度对作为一个二维向量进行旋转，其模长（信息强度）保持不变。
- 不同组的旋转频率不同，提供了互补的多尺度位置信息。

- 最终的点积会综合所有组的贡献，模型通过学习来利用这种重组后的信息。

十八、全文总结与展望

从探究 Attention 机制为何需要位置信息，到一步步揭开 RoPE 通过“旋转”编码相对位置的神秘面纱，我们完成了一次深度旅行。现在，让我们回顾一下 RoPE 的核心精髓：

- 核心目的：解决 Self-Attention 的位置不敏感问题，让模型理解词语的相对顺序。
- 核心思想：通过绝对位置实现相对编码。将词的绝对位置 m 映射为一个旋转角度 $m\theta$ ，对词向量进行旋转。
- 数学实现：利用复数乘法的几何意义（旋转），证明了经过旋转后的两个向量 (q'_m, k'_n) 的点积，其结果仅与它们的相对距离 $(m - n)$ 相关，而与绝对位置 m, n 无关。
- 高维实现：通过分组旋转和多频率设计，将二维旋转思想扩展到高维向量，使得不同维度可以捕捉不同粒度的相对位置信息。
- 实践优势：作为一种无参数、可外推、且不破坏原有语义信息（保范数）的位置编码方案，RoPE 已成为 LLaMA、Qwen、Mistral 等众多主流大语言模型的基石。

RoPE 的成功不仅在于其设计的优雅与高效，更在于它启发了社区对于位置编码的更深层次思考。未来，如何进一步优化其长距离外推能力、如何将其与其他模型结构更好地结合，仍是值得探索的方向。希望本文能为你彻底理解 RoPE 提供坚实的阶梯。

编辑于 2025-12-28 13:38 · 上海

LLM（大型语言模型）



未登录用户

2 条评论

默认 最新